

Performance Testing

Date	29/09/2026
Team ID	SWTID-2026-1297
Project Name	AI FAQ Assistant API
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman / Thunder Client
Type of Testing	API Testing / Functional Testing
Target Module / API	Authentication APIs, FAQ APIs, FAQ Search API, AI FAQ Generation API
Test Environment	Local / Hosted Server
Test Date	29/09/2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Test user registration API	1	—	User should be successfully registered
2	Test user login and authentication	1	—	User should receive authentication access
3	Test FAQ creation and search	1	—	FAQ should be created and searchable
4	Test AI FAQ generation using Gemini	1	—	AI-generated FAQ content should be returned

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	Not measured	N/A	Performance measurement not documented
2	Response Time (Max)	< 5 seconds	Not measured	N/A	Performance measurement not documented
3	Throughput (Req/sec)		Not measured	N/A	Performance measurement not documented
4	Error Rate	< 1%	Not measured	N/A	Performance measurement not documented

5 CPU Utilization not documented	< 80%	Not measured	N/A	Performance measurement
-------------------------------------	-------	--------------	-----	-------------------------

6 Memory Utilization not documented	< 80%	Not measured	N/A	Performance measurement
--	-------	--------------	-----	-------------------------

Step 4: Observations & Analysis

Key Findings:

The API successfully supports user authentication, FAQ CRUD operations, FAQ search, and AI-powered FAQ generation. Manual API testing was performed using Postman / Thunder Client.

Bottlenecks Identified:

No specific performance bottlenecks were identified because detailed load and performance measurements were not documented.

Optimization Steps Taken:

The application uses a modular backend architecture with separate controllers, middleware, routes, models, and Gemini service components. Centralized validation and error handling are also

implemented.

Step 5: Screenshots / Evidence

The screenshot shows the Visual Studio Code interface with the REST Client extension. A POST request is configured to `http://localhost:5000/api/ai/answer`. The request body is a JSON object: `{ "question": "what is mongodb?" }`. The response is a 200 OK status with a size of 287 Bytes and a time of 3.74 s. The response body is a JSON object: `{ "success": true, "answer": "MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents instead of traditional tables and rows. It is designed for high performance, automatic scaling, and handling large volumes of unstructured or semi-structured data." }`. The terminal shows the command `npm start` and the output `ai-faq-assistant-api@1.0.0 start` and `node src/server.js`. The MongoDB connection string is `ac-4ycvnji-shard-00-00.tpnq19b.mongodb.net`. The chat panel on the right shows a message from the AI agent: "Build with Agent. AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase."

The screenshot shows the Visual Studio Code interface with the REST Client extension. A POST request is configured to `http://localhost:5000/api/ai/answer`. The request body is a JSON object: `{ "question": "what is mongodb?" }`. The response is a 200 OK status with a size of 287 Bytes and a time of 3.74 s. The response body is a JSON object: `{ "success": true, "answer": "MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents instead of traditional tables and rows. It is designed for high performance, automatic scaling, and handling large volumes of unstructured or semi-structured data." }`. The terminal shows the command `npm start` and the output `ai-faq-assistant-api@1.0.0 start` and `node src/server.js`. The MongoDB connection string is `ac-4ycvnji-shard-00-00.tpnq19b.mongodb.net`. The chat panel on the right shows a message from the AI agent: "Build with Agent. AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase."

The screenshot shows the VS Code interface with a REST client request in the 'New Request' tab. The request is a POST to 'http://localhost:5000/api/auth/login'. The response is a JSON object with the following structure:

```

{
  "success": true,
  "message": "Login successful",
  "data": {
    "_id": "6ab7642ad6d8e7593a85451d",
    "name": "priya",
    "email": "priya12@gmail.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZyZjc2NDJhZDZkOGU3NTkzYTg1NDUxZCIsIm1ldCI6MTc2MDQwMzc2ZywiZXhwIjoxNzkyOTk1NzM3fQ.9YK4rat33R_pFBCBATPurJSwaekqRgk3a0wM59Thbg"
  }
}

```

The terminal shows the command 'npm start' and the application running on port 5000. The output indicates that the MongoDB connection is successful and the server is running in development mode.